Reasoning Over Time: State Estimation

Lecture 21

CS 4880

Kirk Duran

Last Time: The World Was Frozen

Bayesian networks let the robot reason about uncertainty in a single snapshot:

$$P(X_1, \ldots, X_n) = \prod_i P(X_i \mid Parents(X_i)).$$

But a delivery robot does not live in a snapshot.

- the robot moves;
- pedestrians move;
- wheel odometry drifts;
- sensor readings arrive continuously;
- yesterday's best estimate may be wrong now.

Symbol reminder: $X_1, \ldots, X_n$: random variables; n: number of variables. P: probability; $|$: "given; $Parents(X_i)$: direct causes/inputs of X_i in the network; $\prod_i$: multiply one factor for each variable.

Key idea

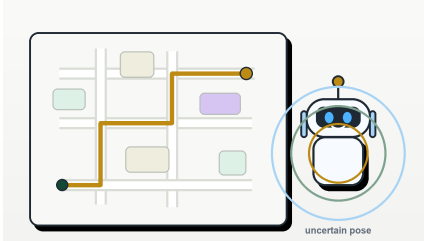
Temporal reasoning asks how an agent should update uncertain beliefs as the hidden world changes over time.

The Campus Robot Now Has a New Problem

The route planner says where the robot *should* be.
The robot still needs to estimate where it *actually* is.

At time t, the robot has:

- a previous belief about its state;
- a control command or odometry estimate;
- a new noisy sensor observation;
- uncertainty in both motion and sensing.

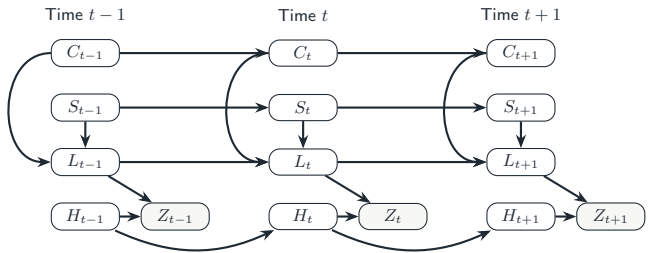


Key idea

Planning chooses a route; state estimation determines where the robot believes it is while executing that route.

A Rich DBN: Many Things Change Together

A dynamic Bayesian network (DBN) repeats a Bayesian network across time. Shaded nodes are sensor readings; the other variables are hidden.



C: crowd conditions; S: slip; L: location; H: sensor health; Z: reading.

Key idea

A DBN can describe a rich changing world, with dependencies within each slice and across time.

One Simple Question Can Require a Lot of Work

Ask: **"Where is the robot now, given all the readings so far?"**

- A surprising reading could mean a different location or an unhealthy sensor.
- An unexpected position could result from wheel slip or a crowd blocking motion.
- Those explanations persist across time and affect one another.

To answer exactly, we must combine evidence and sum over the hidden possibilities. With many variables and time steps, that can take substantial computation.

Naively listing every possible history is especially expensive. Structured inference helps, but a large, tightly coupled hidden state can still be costly.

Key idea

The graph is easy to draw; querying a rich model can be hard. Can we use a simpler model for the question we actually need to answer?

An Easier Starting Point: Track Just the Zone

For a first localization model, keep only

$$X_t \in \{A, B, C\} : \quad \text{the robot's current corridor zone.}$$

- Replace explicit crowd and slip variables with a noisy motion model.
- Replace explicit sensor health with a noisy observation model.
- Assume the current zone and command suffice to predict the next zone.
- Assume the current zone suffices to predict the sensor reading.

This is a simpler model of the world, not just a new name for the rich DBN. Persistent slip or sensor faults may require adding hidden variables back later.

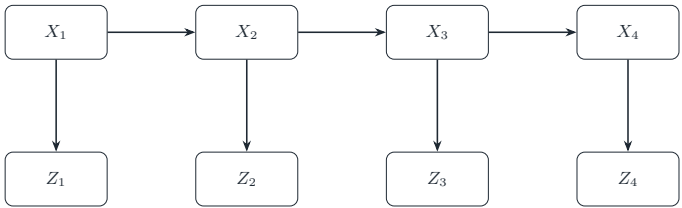
Key idea

Use a small state and explicit independence assumptions, then update a small belief as readings arrive. This gives us a controlled HMM.

A Hidden Markov Model Is a Very Simple DBN

The standard finite-state HMM used here has:

- one finite-valued hidden state variable X_t per time step;
- one observation variable Z_t per time step;
- a first-order Markov transition model;
- a local observation model;
- usually stationary parameters reused through time.



Key idea

HMMs are not a different universe from Bayesian networks; they are a restricted temporal Bayesian-network structure.

The First-Order Markov Assumption

In our simplified model, X_t is the corridor zone. Assume the previous state and current command suffice to predict the next state:

$$P(X_t \mid X_{0:t-1}, U_{1:t}) = P(X_t \mid X_{t-1}, U_t).$$

The past still matters.
It matters *through the current state distribution*.

Symbol reminder: $X_{0:t-1}$: all states from time 0 through $t-1$; $U_{1:t}$: all controls through time t. X_{t-1}: just the previous state; U_t: just the current control; $\mid$: "given.

Key idea

First-order Markov is a modeling assumption shared by many DBNs and HMMs; it does not distinguish the two.

Observations Get a Similar Locality Assumption

For the filtering model used today, assume the observation depends only on the current hidden state:

$$P(Z_t \mid X_{0:t}, Z_{1:t-1}) = P(Z_t \mid X_t).$$

Once X_t is known, earlier states and observations do not directly improve prediction of Z_t.

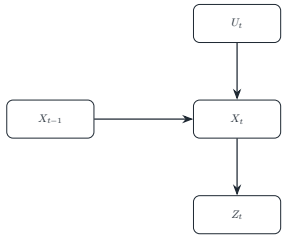
Symbol reminder: Z_t: current observation; X_t: current hidden state; $X_{0:t}$: state history through now; $Z_{1:t-1}$: earlier observations. The equality says that, given X_t, the extra history does not change the observation probability.

Key idea

Observation locality and reusing parameters through time are separate assumptions. A DBN can express other observation dependencies.

For a Robot, Actions Also Affect the Next State

Our simplified hidden state is the zone X_t. A known control U_t, such as `move east`, also affects the next state:



$P(X_t \mid X_{t-1}, U_t)$ motion / transition model

$P(Z_t \mid X_t)$ sensor / observation model

Symbol reminder: X_{t-1}: previous hidden state; X_t: current hidden state; U_t: control for this step; Z_t: current sensor reading. $P(\cdot \mid \cdot)$ means probability given the quantities after the bar.

Key idea

State estimation combines what motion predicts with what the sensors observe.

The Temporal Joint Distribution

Under a first-order transition model and local observation model,

$$P(x_{0:T}, z_{1:T} \mid u_{1:T}) = P(x_0) \prod_{t=1}^{T} P(x_t \mid x_{t-1}, u_t) P(z_t \mid x_t).$$

In this simplified model, every new time step adds:

- one new hidden zone variable;
- one new observation;
- one transition factor;
- one observation factor.

Symbol reminder: $x_{0:T}$: states from time 0 through T; $z_{1:T}$ and $u_{1:T}$: observations and controls from 1 through T. Lowercase letters are particular values of the uppercase random variables. $P(x_0)$: initial belief; $\prod_{t=1}^{T}$: multiply the transition and sensor factors for every step.

Key idea

The model remains locally simple even as the complete trajectory becomes very large.

Temporal Models Support Different Questions

Task	Query	Question
Filtering	$P(X_t \mid z_{1:t})$	Where am I now?
Prediction	$P(X_{t+k} \mid z_{1:t})$	Where might I be later?
Smoothing	$P(X_k \mid z_{1:t}),\ k < t$	Where was I then?
Most likely sequence	$\arg\max x_{1:t} P(x_{1:t} \mid z_{1:t})$	What path most likely occurred?

Today we focus on **filtering / state estimation**.

Symbol reminder: X_t: state now; $z_{1:t}$: all observations so far. In prediction, k is the number of future steps; in smoothing, k is an earlier time. $x_{1:t}$: a candidate state sequence; $\arg\max$: choose the sequence with the highest probability.

Key idea

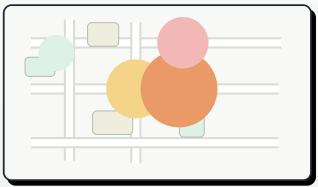
Filtering is online inference: estimate the current hidden state from all evidence observed so far.

Filtering Maintains a Belief State

Define the robot's belief at time t:

$$bel_t(x) = P(X_t = x \mid z_{1:t}, u_{1:t}).$$

This is not one guessed location.
It is a probability distribution over possible current states.



probability mass over locations

Symbol reminder: $bel_t(x)$: probability assigned to candidate state x at time t; $X_t = x$: the hidden state equals that candidate. $z_{1:t}$: observations so far; $u_{1:t}$: controls so far; $|$: "given."

Key idea

State estimation should represent uncertainty, not merely output a single point estimate.

HMM Filtering Is a Two-Step Recursion

1. Predict using motion:

$$\overline{bel}_t(x_t) = \sum_{x_{t-1}} P(x_t \mid x_{t-1}, u_t) bel_{t-1}(x_{t-1}).$$

2. Correct using the new observation:

$$bel_t(x_t) = \eta\, P(z_t \mid x_t) \overline{bel}_t(x_t).$$

η normalizes the distribution.

Symbol reminder: x_t: candidate current state; x_{t-1}: candidate previous state; u_t: current control; z_t: new observation. bel_{t-1}: previous belief; $\overline{bel}_t$: predicted belief before sensing; bel_t: corrected belief. $\sum_{x_{t-1}}$: add over all previous states; η: scale so probabilities sum to 1.

Key idea

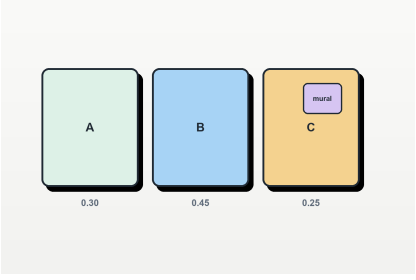
Every filter cycles through the same idea: predict from dynamics, then correct with evidence.

Worked Example: A Three-Zone Campus Corridor

The robot is hidden in one of three zones:

$$X_t \in \{A, B, C\}.$$

It commands `move east`.
The sensor reports whether it sees a distinctive mural near zone C.



Key idea

We will compute one exact filtering update by hand, then examine what happens as the state space and history grow.

Transition Model: Motion Is Not Deterministic

For command `move east`:

	$X_t = A$	$X_t = B$	$X_t = C$
$X_{t-1} = A$	0.20	0.80	0.00
$X_{t-1} = B$	0.00	0.20	0.80
$X_{t-1} = C$	0.00	0.00	1.00

The robot may fail to move because of wheel slip, a pedestrian, or control error.

Key idea

A transition model converts control uncertainty into a distribution over possible next states.

Observation Model: The Mural Sensor Is Noisy

Let $Z_t = +$ mean "mural detected."

State	$P(Z_t = + \mid X_t)$	$P(Z_t = - \mid X_t)$
A	0.05	0.95
B	0.20	0.80
C	0.90	0.10

The sensor is useful, but neither perfect nor deterministic.

Key idea

The observation model tells us how plausible each sensor reading would be from each hidden state.

Start with an Uncertain Prior

Before moving,

$$bel_0 = [0.60,\ 0.30,\ 0.10].$$

So the robot currently believes:

$$P(A) = 0.60, \qquad P(B) = 0.30, \qquad P(C) = 0.10.$$

It now commands `move east`.

Symbol reminder: bel_0: initial belief before moving. Entries are ordered $[A, B, C]$; each entry is the probability of that zone, and the entries sum to 1.

Key idea

Filtering never starts from certainty unless the system genuinely has certainty.

Prediction Step: Push the Belief Through Motion

$$\overline{bel}_1(A) = 0.60(0.20) = 0.12$$
$$\overline{bel}_1(B) = 0.60(0.80) + 0.30(0.20) = 0.54$$
$$\overline{bel}_1(C) = 0.30(0.80) + 0.10(1.00) = 0.34$$

Therefore,

$$\overline{bel}_1 = [0.12,\ 0.54,\ 0.34].$$

Symbol reminder: $\overline{bel}_1$: predicted belief after the first move, before the sensor update; entries are ordered $[A, B, C]$. Each product is previous-zone probability $\times$ transition probability; add all ways to reach the destination zone.

Key idea

Motion spreads probability mass because uncertain actions create several plausible next states.

Correction Step: The Sensor Reports "Mural"

Suppose $Z_1 = +$.
Unnormalized posterior:

$$[0.12(0.05),\ 0.54(0.20),\ 0.34(0.90)]$$
$$= [0.006,\ 0.108,\ 0.306].$$

Normalize by 0.420:

$$bel_1 \approx [0.014,\ 0.257,\ 0.729].$$

Symbol reminder: $Z_1 = +$: the first reading detects the mural. Entries are ordered $[A, B, C]$; each product is predicted-zone probability $\times$ sensor likelihood. 0.420: sum of the unnormalized entries; bel_1: corrected belief after dividing each entry by that sum.

Key idea

Motion favored zone B, but the new sensor evidence shifts most probability mass to zone C.

That Was Easy Because the State Was Tiny

We tracked only three possible states.
A real robot pose might include

$$X_t = (x_t, y_t, \theta_t, v_t, \text{slip}_t, \text{sensorHealth}_t, \ldots).$$

Even if every variable is discretized, the joint state space multiplies:

$$|X| = |x|\,|y|\,|\theta|\,|v|\,|slip| \cdots$$

Symbol reminder: X_t: complete state; x_t, y_t: position coordinates; θ_t: heading; v_t: velocity; slip_t: wheel-slip state; sensorHealth_t: sensor condition. Here $|X|$ means number of joint states, and $|x|, |y|, \ldots$ are counts of possible values for each component.

Key idea

Exact filtering cost depends on the size and structure of the hidden state representation.

If We Track Full Histories, the Number of Paths Explodes

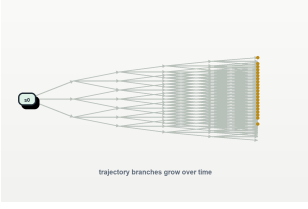
With n possible states per time step, a length-T state history has

$$n^{T+1}$$

possible trajectories.
Example with only 20 possible states and 10 transitions:

$$20^{11} = 2.048 \times 10^{14}$$

possible state sequences.



Key idea

Explicitly retaining every possible history is not an online inference strategy.

Do We Actually Care About the Entire History?

To estimate the robot's *current* state, do we need to remember

$$X_0, X_1, \ldots, X_{t-2}, X_{t-1}$$

individually?
Under our Markov and observation assumptions, the current belief already summarizes the evidence needed for the next update.

Merge histories that reach the same current state.

Key idea

We compress the observation history into a current belief. We have not removed time: the state and its belief still change at every step.

Why This Avoids the History Explosion

Suppose the HMM has n discrete states.
We do *not* retain all n^{T+1} trajectories.
We retain only

$$bel_t = [P(X_t = s_1 \mid z_{1:t}), \ldots, P(X_t = s_n \mid z_{1:t})].$$

A dense transition update costs roughly

$$O(n^2)$$

per time step rather than enumerating all histories.

Symbol reminder: $s_1, \ldots, s_n$: the n possible discrete states; bel_t: their current probability vector; $z_{1:t}$: observations so far. T: number of transitions in a full history; $O(n^2)$: work grows roughly with the square of the state count for a dense transition model.

Key idea

Dynamic programming works because many different histories lead to the same current state and can be merged into one probability mass.

HMM Tradeoff: The State Must Be Good Enough

The Markov assumption is only useful if X_t actually captures the information needed for future prediction.
Bad state choice:

- omit velocity, and position may no longer predict next position well;
- omit sensor health, and observation errors may look inexplicable;
- omit crowd flow, and motion uncertainty may become strongly history-dependent.

Key idea

The price of Markov tractability is a state representation rich enough to make the future approximately conditionally independent of the distant past.

HMMs Are Excellent When the Hidden State Is Discrete

Examples:

- robot is in one of a finite set of rooms;
- machine is in one of several health modes;
- user is in one of several activity states;
- speech system is in one of a sequence of phonetic states.

But our delivery robot's pose is not naturally

$$\{A, B, C\}.$$

It is closer to

$$(x, y, \theta) \in \mathbb{R}^2 \times [-\pi, \pi).$$

Symbol reminder: x, y: position coordinates; θ: heading angle in radians. $\mathbb{R}^2$: any real-valued position pair; $[-\pi, \pi)$: heading range including $-\pi$ and excluding π; $\times$: combine position and heading into one pose.

Key idea

The next question is how to maintain a belief over a continuous state without discretizing the world into millions of bins.

Why Not Just Discretize the Continuous Pose?

Suppose campus localization uses:

- 1000 possible x bins;
- 1000 possible y bins;
- 360 heading bins.

Then the pose grid already contains

$$1000 \times 1000 \times 360 = 360{,}000{,}000$$

discrete states.
Before adding velocity, sensor bias, or anything else.

Key idea

Fine discretization can recover continuous detail, but the state count explodes combinatorially.

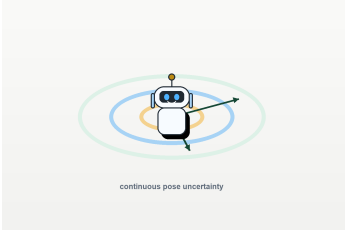
What If the Belief Has a Simple Shape?

Instead of storing every possible continuous state, suppose the belief is Gaussian:

$$X_t \sim \mathcal{N}(\mu_t, \Sigma_t).$$

Then the full belief is summarized by:

- a mean vector μ_t;
- a covariance matrix Σ_t.



Symbol reminder: X_t: continuous hidden state; $\sim$: "is distributed as; $\mathcal{N}$: Gaussian (normal) distribution. μ_t: mean / center; Σ_t: covariance, describing spread and how state components vary together.

Key idea

A parametric filter gains efficiency by assuming the belief can be represented with a small number of parameters.

The Linear-Gaussian State-Space Model

Kalman filtering assumes linear dynamics with Gaussian noise:

$$X_t = AX_{t-1} + BU_t + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, Q).$$

Measurements are also linear with Gaussian noise:

$$Z_t = HX_t + \delta_t, \qquad \delta_t \sim \mathcal{N}(0, R).$$

Symbol reminder: X_{t-1}, X_t: previous/current state; U_t: control; Z_t: measurement. A: state-transition matrix; B: control-effect matrix; H: maps state to measurement. ϵ_t, δ_t: process/sensor noise; Q, R: their covariance matrices. $\mathcal{N}(0, Q)$ and $\mathcal{N}(0, R)$: zero-mean Gaussian noise.

Key idea

Linear transformations of Gaussian variables remain Gaussian, so prediction and correction stay inside the same compact family.

Kalman Filtering Is Still Predict Then Correct

Predict:

$$\mu_t^- = A\mu_{t-1} + BU_t$$
$$\Sigma_t^- = A\Sigma_{t-1}A^T + Q$$

Correct:

$$K_t = \Sigma_t^- H^T (H\Sigma_t^- H^T + R)^{-1}$$
$$\mu_t = \mu_t^- + K_t(Z_t - H\mu_t^-)$$

Symbol reminder: μ, Σ: state mean and covariance; superscript $-$: prediction before sensing, not subtraction; $t-1$: previous step. A: dynamics; B: control mapping; U_t: control; Q: process-noise covariance. H: measurement mapping; R: sensor-noise covariance; Z_t: measurement. K_t: Kalman gain; $Z_t - H\mu_t^-$: measurement minus prediction. Superscript T: matrix transpose; $^{-1}$: matrix inverse.

Key idea

The Kalman gain decides how strongly to trust the new measurement relative to the predicted uncertainty.

One-Dimensional Kalman Example

Suppose before the measurement:

$$X_t \sim \mathcal{N}(12, 5).$$

The position sensor reports

$$z_t = 11, \qquad R = 4.$$

Kalman gain:

$$K = \frac{5}{5+4} = 0.556.$$

Updated mean:

$$\mu_t = 12 + 0.556(11 - 12) \approx 11.44.$$

Updated variance:

$$\Sigma_t = (1 - K)5 \approx 2.22.$$

Symbol reminder: Here $\mathcal{N}(12, 5)$ has predicted mean 12 and variance 5 (not standard deviation). $z_t = 11$: measured position; $R = 4$: sensor variance; K: gain / measurement weight. μ_t: updated mean; Σ_t: updated variance. Direct position sensing uses $H = 1$.

Key idea

The estimate moves toward the measurement, while uncertainty decreases because prediction and sensing provide complementary information.

Why Kalman Filters Are So Attractive

- continuous state without massive discretization;
- compact belief representation;
- recursive online updates;
- mathematically exact for the linear-Gaussian model;
- efficient matrix operations;
- natural fusion of multiple noisy measurements.

Key idea

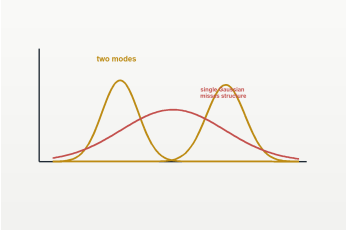
Strong assumptions buy an unusually efficient and elegant state estimator.

But the Gaussian Assumption Can Be the Wrong Shape

Imagine two visually identical campus corridors.
The robot may plausibly be in either one.
The belief is now **multimodal**:

$$p(x) \approx \text{peak near corridor 1} + \text{peak near corridor 2}.$$

One Gaussian tends to summarize that as a mean location *between* the two possibilities.



Symbol reminder: x: a candidate continuous state; $p(x)$: probability density at that state; $\approx$: approximate shape. The two peaks represent two plausible locations, rather than one certain location between them.

Key idea

A compact parameterization can be computationally excellent while representing the wrong uncertainty geometry.

Kalman Failure Modes Motivate a Different Representation

Standard Kalman filtering assumes:

- linear transition model;
- linear observation model;
- Gaussian process noise;
- Gaussian sensor noise;
- effectively unimodal belief.

Real localization can be nonlinear, asymmetric, and multimodal.

Key idea

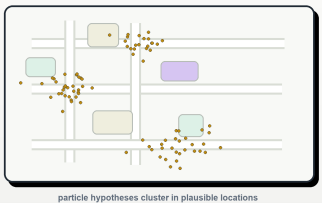
If the posterior cannot be represented well by one Gaussian, we need a belief representation that can take almost any shape.

Lecture 20 Comes Back: Represent the Belief with Samples

A particle filter approximates the current belief with weighted samples:

$$bel_t(x) \approx \{(x_t^{[1]}, w_t^{[1]}), \ldots, (x_t^{[M]}, w_t^{[M]})\}.$$

Each particle is one hypothesis about the current state.
Many particles in one region means high probability mass there.



Symbol reminder: $bel_t(x)$: current belief at candidate state x; $x_t^{[i]}$: state proposed by particle i; $w_t^{[i]}$: its weight; M: number of particles. $[i]$ labels a particle, not a power; $\approx$: the weighted sample set approximates the belief.

Key idea

Particle filtering replaces one analytic distribution with a population of explicit state hypotheses.

Particle Filtering: Predict, Weight, Resample

At each time step:

1. **Predict:** sample each particle through the motion model.
2. **Weight:** score how well each predicted particle explains the observation.
3. **Normalize:** convert scores into a probability distribution.
4. **Resample:** preferentially copy high-weight particles and discard low-weight ones.

$$w_t^{[i]} \propto P(z_t \mid x_t^{[i]}).$$

Symbol reminder: $w_t^{[i]}$: weight of particle i; $x_t^{[i]}$: its predicted state; z_t: new sensor reading. $P(z_t \mid x_t^{[i]})$: how likely that reading is at the particle state; $\propto$: proportional to, before normalizing weights to sum to 1.

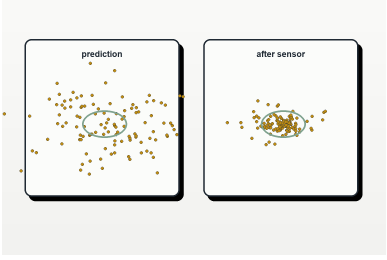
Key idea

Particle filtering is sequential importance sampling plus resampling, applied repeatedly as evidence arrives.

Why Resampling Matters

Without resampling, many particles eventually carry almost no weight.
After resampling:

- hypotheses consistent with the sensor are duplicated;
- inconsistent hypotheses disappear;
- computation is concentrated where posterior probability is high.



Key idea

Resampling reallocates a fixed computational budget toward the parts of state space that currently matter.

Particle Filters Can Represent Multiple Hypotheses

Suppose the robot could be in either of two similar corridors.
A particle set can maintain two clusters simultaneously.
New sensor evidence may later eliminate one cluster.

Kalman belief

One Gaussian center + covariance.

Particle belief

Arbitrary shape, including multiple separated modes.

Key idea

Particle filters trade analytic compactness for representational flexibility.

Particle Filters Have Their Own Failure Modes

- too few particles can miss important regions entirely;
- high-dimensional states may require very many particles;
- repeated resampling can reduce diversity;
- poor motion or sensor models still produce poor estimates;
- computation is stochastic rather than deterministic.

Key idea

Sampling avoids some analytic assumptions, but it pays with Monte Carlo variance and potentially large particle counts.

AMCL: Adaptive Monte Carlo Localization

In ROS 2 Navigation, AMCL is a particle-filter-based localizer for estimating robot pose in a known static map.
It combines:

- a prior / current cloud of pose particles;
- odometry-based motion updates;
- laser-scan likelihoods against the map;
- resampling to concentrate hypotheses;
- an estimated pose distribution for navigation.

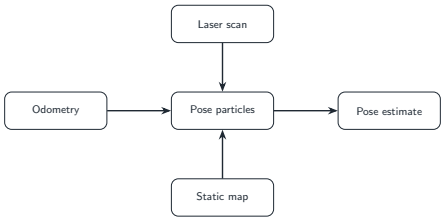
Key idea

AMCL is not just an abstract probability example: it operationalizes particle filtering for practical mobile-robot localization.

What AMCL Is Estimating

The hidden state is approximately robot pose:

$$X_t = (x_t, y_t, \theta_t).$$



Evidence comes from a range sensor scored against a known map; motion information comes from odometry.

Symbol reminder: X_t: robot pose at time t; x_t, y_t: position coordinates in the map; θ_t: heading angle.

Key idea

Localization is filtering: estimate current pose from motion history and sensor evidence without storing every past trajectory.

One Problem, Three Belief Representations

HMM

State: discrete
Belief: probability vector
Exact? yes, under the model
Multimodal? yes, discrete
Cost: state count
Assumption: Markov + finite state

Kalman

State: continuous
Belief: Gaussian (μ, Σ)
Exact? linear-Gaussian
Multimodal? poorly
Cost: matrix algebra
Assumption: linear + Gaussian

Particle filter

State: general / continuous
Belief: weighted samples
Exact? approximate
Multimodal? yes
Cost: particle count
Assumption: representative samples

Key idea

The filtering idea is shared; the algorithms differ mainly in how they represent and update the belief distribution.

The Whole Lecture in One Flow

1. Static Bayesian network: reason about one snapshot.
2. Dynamic Bayesian network: connect factored variable slices through time.
3. Simplify to a small HMM with Markov and local-observation assumptions.
4. Filtering: maintain $P(X_t \mid z_{1:t})$ online using the current belief.
5. Choose a belief representation appropriate to the problem.

HMM	Kalman	Particle filter
Discrete state Probability vector	Continuous state Gaussian belief	General state Weighted samples

Key idea

State estimation is a sequence of tradeoffs: what state to remember, what uncertainty shape to assume, and how much computation to spend.

Takeaways

- Rich DBNs describe interacting variables over time; exact queries can be costly.
- Filtering recursively combines a transition model with new evidence.
- A small state and local assumptions let us update a belief without enumerating histories.
- Finite-state HMMs are a special case of DBNs; forward filtering is exact.
- Kalman filters compress continuous linear-Gaussian beliefs into a mean and covariance.
- Particle filters approximate flexible, multimodal beliefs with weighted samples.
- AMCL applies particle filtering to practical map-based mobile-robot localization.

Key idea

The central engineering question is not "which filter is best?" It is "which representation of uncertainty matches the structure of this state-estimation problem?"